AISC5020
Week 7
Statistical Inference

Presented by

Simon Cheung

Email: kingchaucheung@cuhk.edu.hk

Department of Statistics and Data Science,
The Chinese University of Hong Kong

Chapter Outline

- Likelihood and loglikelihood
- Least squares and MLE
- MSE loss and its gradient
- MAP estimation and regularization

Likelihood and logLikelihood

- Suppose $\{X_1, \dots, X_n\}$ is a IID random sample. Let $f_\theta : \mathcal{X} \to \mathbb{R}$ be the pdf or pmf of X. θ is an unknown parameter that characterizes the distribution of X.
- The joint pdf or pmf of $(X_1, \dots, X_n)$ is

$$f_\theta(x_1, \dots, x_n) = \prod_{j=1}^{n} f_\theta(x_j)$$

- This is a function of $(x_1, \dots, x_n)$ for a given θ.
- Suppose a sample is observed, that is $(X_1, \dots, X_n) = (x_1, \dots, x_n)$. We can view $f_\theta(x_1, \dots, x_n)$ as a function of θ for a given observed sample $(x_1, \dots, x_n)$. We call it the likelihood function for θ based on the observed sample $(x_1, \dots, x_n)$.

$$L(\theta) = \prod_{j=1}^{n} f_\theta(x_j)$$

$$\ell(\theta) = \ln L(\theta) = \sum_{j=1}^{n} \ln f_\theta(x_j)$$

Likelihood and logLikelihood

- Example. Let $\{X_1, \ldots, X_n\}$ be an independent random sample from $n(\mu, \sigma^2)$, where $(\mu, \sigma^2) \in \Omega$ and $\Omega = \{(\mu, \sigma^2): \mu \in \mathbb{R}, \sigma^2 > 0\}$.

- The likelihood function for (μ, σ^2) based on the observed sample $\{x_1, \ldots, x_n\}$ is given by

$$L(\mu, \sigma^2) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_i - \mu)^2}{2\sigma^2}\right].$$

- The log-likelihood function for (μ, σ^2) is

$$\ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2 .$$

Likelihood and logLikelihood

- Example. Suppose that X is a Bernoulli random variable. Its pdf is $f(x;p) = p^x(1-p)^{1-x}, x \in \{0,1\}$. Here, the unknown parameter is $p \in \Omega = [0,1]$. Assume that a random sample $\{X_1, \dots, X_n\}$ with the observed values $\{x_1, \dots, x_n\}$ is obtained. Then, the probability $P(X_1 = x_1, \dots, X_n = x_n; p)$ can be evaluated as

$$P(X_1 = x_1, \dots, X_n = x_n; p) = \prod_{j=1}^{n} p^{x_i}(1-p)^{1-x_i} = p^{\sum_{i=1}^n x_i}(1-p)^{n-\sum_{i=1}^n x_i}$$

- Consider that the random sample is fixed at the observed values, the joint probability is a function of p. When we consider the joint probability as a function of p, we write

$$L(p|x_1, \dots, x_n) = P(X_1 = x_1, \dots, X_n = x_n; p) = p^{\sum_{i=1}^n x_i}(1-p)^{n-\sum_{i=1}^n x_i}$$

$$\ell(p|x_1, \dots, x_n) = \left(\sum_{i=1}^{n} x_i\right) \ln p + \left(n - \sum_{i=1}^{n} x_i\right) \ln(1-p)$$

- $L(p|x_1, \dots, x_n)$ is called the likelihood of observing $X_1 = x_1, \dots, X_n = x_n$ for the given parameter value p.
- $\ell(p|x_1, \dots, x_n)$ is the loglikelihood of observing $X_1 = x_1, \dots, X_n = x_n$ for the given parameter value p.

Maximum Likelihood Estimation

- Definition. Given a likelihood function $L(\theta)$, $\theta \in \Omega$, based on the observed sample $(x_1, \ldots, x_n)$.
- The **maximum likelihood estimator** (MLE) of θ is defined as

$$\hat{\theta} = \arg\max_{\theta \in \Omega} L(\theta | x_1, \ldots, x_n) = \arg\max_{\theta \in \Omega} \ell(\theta | x_1, \ldots, x_n) .$$

- The observed value of $\hat{\theta}$ is called the maximum likelihood estimate of θ.
- Example. Let X be an independent random sample from a uniformly distribution over the interval $[0, \beta]$. Find the maximum likelihood estimator of β.
- The uniform distribution over $[0, \beta]$ has the pdf $f(x; \beta) = \frac{1}{\beta} I(0 \le x \le \beta)$.
- For an observed sample $\{x_1, \ldots, x_n\}$, the likelihood function is

$$L(\beta) = \prod_{i=1}^{n} \frac{1}{\beta} I(0 \le x_i \le \beta) = \frac{1}{\beta^n} \prod_{i=1}^{n} I(0 \le x_i \le \beta) = \frac{1}{\beta^n} I\left(\max_{1 \le i \le n} x_i \le \beta\right)$$

- From the graph, we see that $\hat{\beta} = \max_{1 \le i \le n} x_i = x_{(n)}$.

Maximum Likelihood Estimation

- Example. Let $\{X_1, \ldots, X_n\}$ be an independent random sample from $n(\mu, \sigma^2)$, where $(\mu, \sigma^2) \in \Omega$ and $\Omega = \{(\mu, \sigma^2): \mu \in \mathbb{R}, \sigma^2 > 0\}$. Find the MLEs of (μ, σ^2).

- The log-likelihood function is

$$\ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2 = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{j=1}^{n}\left(x_j - \bar{x}\right)^2 - \frac{n}{2\sigma^2}(\bar{x} - \mu)^2$$

- The MLE $(\hat{\mu}, \hat{\sigma}^2)$ of $\ell(\mu, \sigma^2)$ should satisfy

$$0 = \frac{\partial \ell(\hat{\mu}, \hat{\sigma}^2)}{\partial \mu} = \frac{1}{\hat{\sigma}^2}(\bar{x} - \hat{\mu})$$

$$0 = \frac{\partial \ell(\hat{\mu}, \hat{\sigma}^2)}{\partial \sigma^2} = -\frac{n}{2\hat{\sigma}^2} + \frac{1}{2\hat{\sigma}^4}\sum_{i=1}^{n}(x_i - \hat{\mu})^2$$

- Solving the two equations, we obtain

$$\hat{\mu} = \bar{x}_n \text{ and } \hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x}_n)^2$$

Maximum Likelihood Estimation

- Let $\{X_1, \ldots, X_n\}$ be an independent random sample from a distribution with pdf $f(x;\theta)$. Suppose that $\hat{\theta}$ is an estimator of θ. If $E(\hat{\theta}) = \theta$, we call $\hat{\theta}$ an unbiased estimator of θ and the quantity $E(\hat{\theta}) - \theta$ is called the **Bias** of $\hat{\theta}$. When the bias of $\hat{\theta}$ is not zero, $\hat{\theta}$ is a biased estimator of θ.

- $\hat{\theta}$ is an asymptotically unbiased estimator of θ if

$$\lim_{n\to\infty} Bias(\hat{\theta}) = \lim_{n\to\infty} \{E(\hat{\theta}) - \theta\} = 0,$$

- where n is the sample size.
- Suppose that $\hat{\theta}$ is an estimator of θ. The **mean squared error** of $\hat{\theta}$ is $MSE(\hat{\theta}) = E\left[(\hat{\theta} - \theta)^2\right]$.
- If $Var(\hat{\theta})$ exists, we have $MSE(\hat{\theta}) = Var(\hat{\theta}) + \left[Bias(\hat{\theta})\right]^2$.
- Proof.

$$MSE(\hat{\theta}) = E\left[(\hat{\theta} - \theta)^2\right] = E\left\{\left[\left(\hat{\theta} - E(\hat{\theta})\right) + \left(E(\hat{\theta}) - \theta\right)\right]^2\right\}$$
$$= E\left\{\left(\hat{\theta} - E(\hat{\theta})\right)^2 + \left(E(\hat{\theta}) - \theta\right)^2 + 2\left(\hat{\theta} - E(\hat{\theta})\right)\left(E(\hat{\theta}) - \theta\right)\right\} = Var(\hat{\theta}) + \left[E(\hat{\theta}) - \theta\right]^2$$

Maximum Likelihood Estimation

- Example. Suppose that $\{X_1, \dots, X_n\}$ is a random sample from a distribution with pdf

$$f(x, \delta) = e^{-(x-\delta)}, x \geq \delta.$$

- Show that $\bar{X}_n$ is a biased estimator of δ. Find the mean squared error of $\bar{X}_n$.

- Let $Y = X - \delta$. Then the pdf of Y is given by $g(y) = e^{-y}, y \geq 0$.

- Since $E(Y) = 1$,

$$E(X) = 1 + \delta \Longrightarrow E(\bar{X}_n) = 1 + \delta.$$

$$Var(X) = Var(Y) = 1 \Longrightarrow Var(\bar{X}_n) = \frac{1}{n}.$$

$$MSE(\bar{X}_n) = Var(\bar{X}_n) + [E(\bar{X}_n) - \delta]^2 = 1 + \frac{1}{n}.$$

Maximum Likelihood Estimation

- Example. Let X be an independent random sample from a uniformly distribution over the interval $[0, \beta]$. $\hat{\beta} = X_{(n)}$ is the maximum likelihood estimator of β. Show that $\hat{\beta}$ is an asymptotically unbiased estimator of β and modify $\hat{\beta}$ to make it unbiased.

- Note that

$$P\left(X_{(n)} \leq x\right) = \prod_{i=1}^{n} P(X_i \leq x) = \left(\frac{x}{\beta}\right)^n.$$

$$E\left(X_{(n)}\right) = \int_0^{\beta} P\left(X_{(n)} > x\right) dx = \int_0^{\beta} \left\{1 - \left(\frac{x}{\beta}\right)^n\right\} dx = \beta - \frac{\beta^{n+1}}{(n+1)\beta^n} = \frac{n\beta}{n+1} \longrightarrow \beta,$$

- as $n \to \infty$.
- Since $E\left(X_{(n)}\right) = \frac{n\beta}{n+1}$, we see that $E\left(\frac{n+1}{n} X_{(n)}\right) = \beta$. Hence, $\frac{n+1}{n} X_{(n)}$ is an unbiased estimator of β.

Maximum Likelihood Estimation

- Example. Let X be an independent random sample from a uniformly distribution over the interval $[0, \beta]$. $\frac{n+1}{n} X_{(n)}$ is an unbiased estimator of β. Show that
  - $2\bar{X}$ is also an unbiased estimator of β, and Compare the efficiency of these two estimators of β.

- Since X is uniformly distributed on $[0, \beta]$, $E(X) = \frac{\beta}{2}$ and $Var(X) = \frac{\beta^2}{12}$. We have $E(\bar{X}) = \frac{\beta}{2}$. Hence, $2\bar{X}$ is an unbiased estimator of β. Moreover,

$$Var(2\bar{X}) = 4Var\left(\frac{1}{n}\sum_{j=1}^{n} X_j\right) = \frac{4}{n^2}\frac{n\beta^2}{12} = \frac{\beta^2}{3n}.$$

- Since $P\left(X_{(n)} \le x\right) = \left(\frac{x}{\beta}\right)^n, x \in [0, \beta]$,

$$P\left(X_{(n)}^2 \le z\right) = P\left(X_{(n)} \le \sqrt{z}\right) = \left(\frac{\sqrt{z}}{\beta}\right)^n, z \in [0, \beta^2].$$

$$E\left(X_{(n)}^2\right) = \int_0^{\beta^2} P\left(X_{(n)}^2 > z\right) dz = \int_0^{\beta^2} \left[1 - \left(\frac{\sqrt{z}}{\beta}\right)^n\right] dz = 2\int_0^{\beta} y - \frac{y^{n+1}}{\beta^n} dy = 2\left[\frac{\beta^2}{2} - \frac{1}{\beta^n}\frac{\beta^{n+2}}{n+2}\right] = \frac{n}{n+2}\beta^2$$

Maximum Likelihood Estimation

$$Var\left(\frac{n+1}{n}X_{(n)}\right) = \frac{(n+1)^2}{n^2}Var\left(X_{(n)}\right) = \frac{(n+1)^2}{n^2}\left[\frac{n}{n+2}\beta^2 - \frac{n^2}{(n+1)^2}\beta^2\right] = \frac{(n+1)^2}{n^2}\frac{n\beta^2}{(n+2)(n+1)^2}$$
$$= \frac{\beta^2}{n(n+2)}.$$

- Therefore,

$$Eff\left(\frac{n+1}{n}X_{(n)}, 2\bar{X}\right) = \frac{Var(2\bar{X})}{Var\left(\frac{n+1}{n}X_{(n)}\right)} = \frac{\frac{\beta^2}{3n}}{\frac{\beta^2}{n(n+2)}} = \frac{n+2}{3} > 1.$$

- Thus, $\frac{n+1}{n}X_{(n)}$ is more efficient than $2\bar{X}$.

Remark: The MLE of θ is

$$\hat{\theta} = \underset{\theta\in\Omega}{\operatorname{argmax}}\,\ell(\theta) = \underset{\theta\in\Omega}{\operatorname{argmax}}\sum_{j=1}^{n}\ln f_\theta(x_j) = \underset{\theta\in\Omega}{\operatorname{argmax}}\frac{1}{n}\sum_{j=1}^{n}\ln f_\theta(x_j) = \underset{\theta\in\Omega}{\operatorname{argmax}}\,E[\ln f_\theta(X)]$$

MLE and KL Divergence

- Suppose we want to make inference on the true data generating distribution f. We use a family of distributions $\{f_\theta : \theta \in \Omega\}$ to model or to learn f.

- Essentially, we are trying to estimate f with our model distributions by finding good estimates to θ.

$$\underset{\theta\in\Omega}{\operatorname{argmin}} D_{f_\theta}(f) = \underset{\theta\in\Omega}{\operatorname{argmin}} E_f\left(\log_2\left(\frac{f(X)}{f_\theta(X)}\right)\right) = \underset{\theta\in\Omega}{\operatorname{argmax}} E_f(\log_2 f_\theta(X)) = \hat{\theta}.$$

- Finding MLE is minimizing the KL Divergence between the true distribution and the model distribution.

Conditional Loglikelihood

- Consider a dataset of n samples $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ drawn independently IID from the true but unknown data generating distribution $P_\theta(x, y)$, where $(x, y) \in \mathbb{R}^k \times \mathcal{Y}$.
- To predict y given x, we need to estimate the conditional probability $P_\theta(y|x)$.
- The conditional maximum likelihood estimator for θ based on the observed sample is

$$\hat{\theta} = \underset{\theta \in \Omega}{\operatorname{argmax}} \prod_{i=1}^{n} P_\theta(y_i|x_i) = \underset{\theta \in \Omega}{\operatorname{argmax}} \sum_{i=1}^{n} \ln P_\theta(y_i|x_i)$$

Conditional Loglikelihood

- In a linear regression problem, we assume that $y = x'\beta + \varepsilon, \varepsilon \sim n(0, \sigma^2)$.

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^{k+1}}{\operatorname{argmax}} \left(-\frac{n}{2} \ln \sigma^2 - \frac{1}{2\sigma^2} \sum_{j=1}^{n} \left(y_j - x_j' \beta \right)^2 \right) = \underset{\beta \in \mathbb{R}^{k+1}}{\operatorname{argmin}} \sum_{j=1}^{n} \left(y_j - x_j' \beta \right)^2$$

- The minimizing function $\frac{1}{n} \sum_{j=1}^{n} \left(y_j - \hat{y}_j \right)^2$, where $\hat{y}_j = x_j' \beta$ is the predicted value for y_j, is called the mean squared error loss function.
- Since $\sum_{j=1}^{n} \left(y_j - x_j' \beta \right)^2$ is the sum of error squared, $\hat{\beta}$ is also called the linear least square error estimator of β.

Conditional Loglikelihood

- In a logistic regression problem, we assume that $y_i \in \{0,1\}$ and

$$P(y|x,\beta) = \left(\frac{e^{x'\beta}}{1+e^{x'\beta}}\right)^{y}\left(\frac{1}{1+e^{x'\beta}}\right)^{1-y} = \frac{e^{x'\beta y}}{1+e^{x'\beta}}, y \in \{0,1\}$$

- The likelihood for β for observing the sample $\{(x_1,y_1),(x_2,y_2),\dots,(x_n,y_n)\}$ is

$$L(\beta) = \prod_{j=1}^{n} P\left(y_j|x_j,\beta\right)$$

$$\ell(\beta) = \ln L(\beta) = \sum_{j=1}^{n}\left(y_j \ln p_j + \left(1-y_j\right)\ln\left(1-p_j\right)\right),$$

- where $p_j = \frac{e^{x_j'\beta}}{1+e^{x_j'\beta}}$.

Conditional Loglikelihood

- The maximum likelihood estimator for β is

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^{k+1}}{\operatorname{argmax}} \sum_{j=1}^{n} \left(y_j \ln p_j + (1 - y_j) \ln(1 - p_j) \right) = \underset{\beta \in \mathbb{R}^{k+1}}{\operatorname{argmin}} - \sum_{j=1}^{n} \left(y_j \ln p_j + (1 - y_j) \ln(1 - p_j) \right)$$

$$= \underset{\beta \in \mathbb{R}^{k+1}}{\operatorname{argmin}} \sum_{j=1}^{n} \left(y_j \ln \frac{1}{p_j} + (1 - y_j) \ln \frac{1}{1 - p_j} \right)$$

- Finding a MLE is equivalent to minimizing the cross entropy of the true (empirical) distribution with respect to the model distribution.

Loss function - MSE

- Let $X \in \mathbb{R}^k$ denote a real-valued random input vector, and $Y \in \mathbb{R}$ a real-valued random output variable, with a joint distribution $P(X, Y)$. Note that the use of distribution applies to both continuous and discrete variables.
- We seek a function $f(X)$ for prediction Y given values of X.
- Define the squared error loss $L\big(Y, f(X)\big) = \big(Y - f(X)\big)^2$.
- Therefore, we choose f to minimize the expected squared (L_2) error loss

$$EL(f) = E\big(Y - f(X)\big)^2 = E_X\left[E_{Y|X}\left(\big(Y - f(X)\big)^2 \middle| X\right)\right]$$

- Since $E_X(\cdot)$ is a weighted average of whatever is inside the bracket,

$$f(x) = \underset{c}{\operatorname{argmin}}\, E_{Y|X}((Y - c)^2 | X = x) = E_{Y|X}(Y | X = x).$$

Loss function - MSE

$$f(x) = \underset{c}{\operatorname{argmin}}\, E_{Y|X}((Y-c)^2|X=x) = \underset{c}{\operatorname{argmin}} E_{Y|X}\left(\left(Y - E_{Y|X}(Y|X=x) + E_{Y|X}(Y|X=x) - c\right)^2 \middle| X=x\right)$$

$$= \underset{c}{\operatorname{argmin}}\, E_{Y|X}\left(\left(c - E_{Y|X}(Y|X=x)\right)^2 - 2\left(c - E_{Y|X}(Y|X=x)\right)\left(Y - E_{Y|X}(Y|X=x)\right) + \left(Y - E_{Y|X}(Y|X=x)\right)^2 \middle| X=x\right)$$

$$= \underset{c}{\operatorname{argmin}} \left(\left(c - E_{Y|X}(Y|X=x)\right)^2 + E_{Y|X}\left(\left(Y - E_{Y|X}(Y|X=x)\right)^2 \middle| X=x\right)\right) = E_{Y|X}(Y|X=x).$$

- The conditional expectation $f(x) = E_{Y|X}(Y|X=x)$ is called the regression function.
  - linear regression $f(x) = x'\beta$ assumes that the regression function is linear.
  - k-nearest neighbours $f(x) = \text{Average}(y_i | x_i \in N_k(x))$ assumes that the regression function is well approximated by locally constant functions.

Loss function - MSE

- The linear regression model $Y = X\beta + \varepsilon, \varepsilon \sim n(0, \sigma^2 I_n)$ has the loglikelihood function

$$\ell(\beta, \sigma^2) = -\frac{n}{2}\ln\sigma^2 - \frac{1}{2\sigma^2}(Y - X\beta)'(Y - X\beta)$$

- The MLE for β is

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^{k+1}}{\operatorname{argmin}}(Y - X\beta)'(Y - X\beta) = (X'X)^{-1}X'Y$$

- Since $(Y - X\beta)'(Y - X\beta)$ is the sum of squared errors, $\hat{\beta}$ is also called the least squared errors estimator and the loss function is $MSE = \frac{1}{n}\sum_{j=1}^{n}\left(y_j - x_j'\beta\right)^2$, the mean squared error loss function.
- MSE is continuous, differentiable and convex (local minimum = global minimum). But it is highly sensitive to outliers.

Loss function - RMSE

- Directly connected to MSE is the Root Mean Squared Error loss (RMSE).

$$RMSE = \sqrt{\frac{1}{n}\sum_{j=1}^{n}\left(y_j - f\left(x_j\right)\right)^2}$$

- The main advantage is that it has the same unit and scale of the target variable.
- Although the minimization process converges to the same optimal value, the RMSE may take different gradient steps.

Loss function - MAE

- Suppose we choose f to minimize the expected absolute error (L_1) loss $EL(f) = E|Y - f(X)|$. We have $f(x) = \underset{c}{\text{argmin}}\, E_{Y|X}(|Y - c||X = x) = \text{median}(Y|X = x)$.

$$g(c) = \int_{-\infty}^{\infty} |y - c| f_{Y|X}(y|x)dy = \int_{-\infty}^{c} (c - y)\, f_{Y|X}(y|x)dy + \int_{c}^{\infty} (y - c) f_{Y|X}(y|x)dy$$

$$= c\int_{-\infty}^{c} f_{Y|X}(y|x)dy - \int_{-\infty}^{c} y f_{Y|X}(y|x)dy + \int_{c}^{\infty} y f_{Y|X}(y|x)dy - c\int_{c}^{\infty} f_{Y|X}(y|x)dy$$

$$\frac{dg(c)}{dc} = \int_{-\infty}^{c} f_{Y|X}(y|x)dy + c f_{Y|X}(c|x) - c f_{Y|X}(c|x) - c f_{Y|X}(c|x) - \int_{c}^{\infty} f_{Y|X}(y|x)dy + c f_{Y|X}(c|x)$$

$$= P(Y \le c|X = x) - P(Y \ge c|X = x)$$

To find c which minimizes $g(c)$, we set $\frac{dg(c)}{dc} = 0 \Leftrightarrow P(Y \le c|X = x) = P(Y \ge c|X = x) = 0.5$. Thus, $f(x) = \text{median}(Y|X = x)$

Loss function - MAE

- The mean absolute error is defined as $MAE = \frac{1}{n}\sum_{j=1}^{n}\left|y_j - f\left(x_j\right)\right|$. It is a continuous and convex function which satisfies the Lipschitz condition, but it is not differentiable at 0.
- It distributes errors linearly, that is doubling an error doubles the penalty.
- Because $\frac{d\,|y-c|}{dc} = \begin{cases} -1 \text{ if } c > y \\ 1 \quad \text{if } c < y \end{cases}$, the gradient descent takes constant step size regardless of how close it is to the target (Large and small errors contribute the same step size).
- Because of constant step size, the gradient descent algorithm might have a convergence problem (fluctuation around zero) or might be slow to converge.
- To handle convex functions with sharp, non-differentiable corners, some frameworks like PyTorch identify the range of valid slopes from lines that touch the corner point without intersecting the function's interior. A slope is then randomly selected from this range.

Loss function - logcosh

- The log-cosh loss is a hybrid between L_2 loss and L_1 loss.
- It is defined as $\text{logcosh} = \frac{1}{n}\sum_{j=1}^{n} \ln\left(\cosh\left(y_j - f\left(x_j\right)\right)\right)$, where $\cosh(x) = \frac{e^x + e^{-x}}{2}$. The loss function is continuous, differentiable and convex everywhere.
- For small errors $|x|$ is small, $\ln\cosh(x) \approx \ln\left(1 + \frac{x^2}{2}\right) \approx \frac{x^2}{2}$. Thus, log-cosh is close to L_2 loss.
- For positive large errors $(x \to \infty)$, $\ln\cosh(x) \approx \ln\left(\frac{e^x}{2}\right) = x - \ln 2$.
- For negative large errors $(x \to -\infty)$, $\ln\cosh(x) \approx \ln\left(\frac{e^{-x}}{2}\right) = -x - \ln 2$.
- To summarize, $\ln\cosh(x) \approx |x| - \ln 2$ for large errors. Thus, log-cosh is close to L_1 loss.
- Since $\frac{d\ln\cosh(x)}{dx} = \tanh x$ and $|\tanh x| < 1$ for all x and $\tanh x \approx x$ for small x, it ensures outliers can never generate large gradients and the gradient descent algorithm converges smoothly to the global minimum.

Loss function - Huber

- A piece-wise hybrid between L_2 loss and L_1 loss is the Huber loss.
- For a single data point $x = |y - f(x)|$, $\text{Huber}(x) = \begin{cases} \frac{1}{2}x^2 & \text{for } |x| < \delta \\ \delta\left(|x| - \frac{\delta}{2}\right) & \text{for } |x| > \delta \end{cases}$.
- For ordinary errors, the Huber loss behaves like MSE.
- For large errors, the Huber loss transitions to a linear form scaled by δ, with a derivative of $\pm\delta$, effectively capping the gradient for large deviations.
- The parameter δ acts as a tuning knob that determines what the model considers an outlier. Because selecting a fixed optimal value is difficult, modern approaches may adjust δ dynamically during training. Early on, a larger δ allows faster, MSE-like updates, while later it can be reduced to better suppress noise and minor outliers as errors decrease.
- The smooth L_1 loss assumes $\delta = 1$ has wide applications in computer vision and object detection models such as Faster R-CNN.

Classification Models

Classification models can produce two main types of outputs:

- **Hard (Discrete) Outputs**
  - These are binary indicators (0 or 1) for each class. So, $G: \mathcal{X} \to \{0,1\}^K$ where K is the number of classes.
  - In **multi-label classification**, multiple entries can be 1 (e.g., an image labeled both "dog" and "tree"). That is the model can predict a vector like $[0,1,0,1,0,0]$.
  - In **single-label classification**, only one entry can be 1 (a one-hot vector), meaning the model must choose exactly one class. We add the constraint that the vector elements must sum to 1.
- **Soft (Probabilistic) Outputs**
  - Most modern models output probabilities instead of hard labels because probabilities are smooth and differentiable. So, $G: \mathcal{X} \to [0,1]^K$.
  - Each value represents $P(Y = k \mid X = x)$, the likelihood of each class.
  - For single-label tasks, probabilities must sum to 1, typically enforced using **Softmax**.

Classification Models

Binary Classification and Margins

- For binary problems, labels are often encoded as -1 and $+1$. This enables the concept of a **margin**:

$$\text{margin} = y \cdot f(x)$$

  where $y \in \{-1,1\}$ is the true label and $f(x) \in \mathbb{R}$ is the raw continuous score (logit) output by the model (where the sign tells you the class, and the magnitude tells you the confidence).

  - If the margin is **positive**, the prediction is correct.
  - If the margin is **negative**, the prediction is wrong.
  - Larger positive margins (value of $f(x)$) indicate more confident correct predictions.
- Many loss functions are designed to **maximize this margin**, not just achieve correctness.

Classification Models

Main Types of Classification Loss Functions

A. Margin-Based Losses – These focus on the distance from the decision boundary.

- **0–1 Loss**: simplest but not usable for optimization
- **Hinge Loss**: penalizes points inside or beyond the margin (used in SVMs)
- **Smoothed variants**: make optimization easier

B. Probabilistic Losses – These treat classification as matching predicted probabilities to true labels.

- **Cross-Entropy / NLL**: strongly penalize assigning low probability to the correct class

Loss function – 0-1 Loss

- When performing classification (e.g., labelling an image as a cat, dog, or bird), we consider a random variable Y representing the true class label, which can take values in $\{1,2,\dots,K\}$ where K is the number of classes. To evaluate predictions, we use a loss matrix L of size $K \times K$.
  - The diagonal entries satisfy $L(k,k) = 0$, since correctly predicting class k incurs no penalty.
  - The off-diagonal entries satisfy $L(k,\ell) > 0$ reflecting a positive penalty for misclassifying class k as ℓ.
- This setup allows for asymmetric penalties. For example, in medical diagnosis, misclassifying a healthy patient as sick may incur a small cost, while misclassifying a sick patient as healthy may incur a much larger cost.
- If all errors are treated equally, this reduces to the 0–1 loss, where each misclassification costs one unit.

Loss function – 0-1 Loss

- The optimal classifier is chosen pointwise by selecting the class $g \in \{1,2,\dots,K\}$ that minimizes the expected loss given the input x:

$$G(x) = \underset{g\in\{1,2,\dots,K\}}{\operatorname{argmin}} \sum_{k=1}^{K} L(k,g)P(Y=k|X=x)$$

- Suppose $L(k,g) = 1$ if $k \neq g$ and $L(k,g) = 0$ if $k = g$.

$$\sum_{k=1}^{K} L(k,g)P(Y=k|X=x) = \underbrace{L(g,g)}_{0}P(Y=g|X=x) + \sum_{\substack{k=1\\k\neq g}}^{K} \underbrace{L(k,g)}_{1}P(Y=k|X=x)$$

$$= \sum_{\substack{k=1\\k\neq g}}^{K} P(Y=k|X=x) = 1 - P(Y=g|X=x).$$

- This gives rise to the Bayes Classifier. But implementation requires the knowledge of $P(Y=g|X=x)$.

Loss function – 0-1 Loss

- The k-nearest neighbor (k-NN) classifier draws a localized circle around a new point x containing its k closest neighbors. It calculates the proportion of each class g inside that circle and uses that proportion as an empirical estimate of $P(Y = g|X = x)$. By running a majority vote, k-NN directly approximates the exact pointwise argmax choice of the Bayes classifier.

Loss function – 0-1 Loss

- Suppose you have a binary classification problem (e.g., Email vs. Spam), where Email is coded as $Y = 0$ and Spam as $Y = 1$. If you fit a regression model using squared error loss, the optimal prediction is the conditional expectation:

$$\hat{f}(x) = E(Y|X = x).$$

- Since Y only takes values 0 or 1, this simplifies to:

$$E(Y|X = x) = \Pr(Y = 1|X = x).$$

- So, regression on 0/1 labels directly estimates the probability that $Y = 1$. Classifying observations with a cutoff of 0.5 then follows the Bayes decision rule.

- According to the Bayes decision rule, we should choose Class 1 if

$$P(Y = 1|X = x) > P(Y = 0|X = x) = 1 - P(Y = 1|X = x) \Longrightarrow P(Y = 1|X = x) > 0.5$$

- However, standard linear regression has a key problem: its predictions are unbounded. It can produce values below 0 or above 1 (e.g., −0.4 or 1.3), which are not valid probabilities.
- This limitation is why linear regression is unsuitable for classification, and why models like logistic regression or neural networks are used instead—they ensure outputs stay between 0 and 1.

Loss function – Hinge Loss

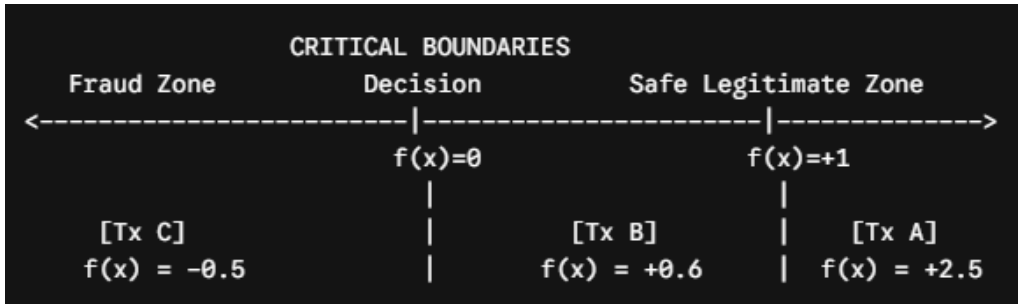
- In classification, our true goal is to minimize the **0–1 loss** (the number of wrong predictions). However, this loss is not smooth (derivative of zero everywhere), so it cannot be optimized with gradient descent.
- A **surrogate loss** is a smooth, easy-to-optimize substitute. By minimizing it (e.g., hinge loss), we indirectly reduce classification errors.
- Hinge loss is defined as:

$$L(f(x), y) = \max\big(0, 1 - yf(x)\big), y \in \{-1, 1\}, f(x) \in \mathbb{R}$$

  where $yf(x)$ is the **margin**.
- It creates two regions:
  - **Safe zone ($yf(x) > 1$)**: prediction is correct and confident → loss = 0
  - **Penalty zone ($yf(x) \leq 1$)**: either wrong or too close to the boundary → loss increases linearly
- This encourages models (like SVMs) to not just be correct, but to stay **far from the decision boundary**.

Loss function – Hinge Loss

- Example: Classify valid $(Y = 1)$ or Fraudulent $(Y = -1)$ bank transactions. Let X be a vector of features. For example, $X =$ (Transaction Amount ($), Time of Day (Hour), Distance from user's home (miles), number of failed login attempts, etc)
- The model calculates a score $f(X = x) = w'x + b$
  - If $f(x) > 0$, the model predicts $Y = 1$.
  - If $f(x) < 0$, the model predicts $Y = -1$.
- The decision boundary is where $f(x) = 0$.



- The margin are located at $f(x) = +1$ and $f(x) = -1$.
- Case A: $f(x) = +2.5$ and True label $Y = 1$. The Hinge loss $L(f(x), y) = \max(0, 1 - 2.5) = 0$ (no loss).
- Case B: $f(x) = +0.6$ and True label $Y = 1$. The Hinge loss $L(f(x), y) = \max(0, 1 - 0.6) = 0.4$ (medium loss).
- Case C: $f(x) = -0.5$ and True label $Y = 1$. The Hinge loss $L(f(x), y) = \max(0, 1 + 0.5) = 1.5$ (large loss).

Loss function – Perceptron Loss

- The Perceptron is one of the earliest algorithmic building blocks of neural networks. Its loss function is defined as

$$L(f(x), y) = \max\big(0, -yf(x)\big)$$

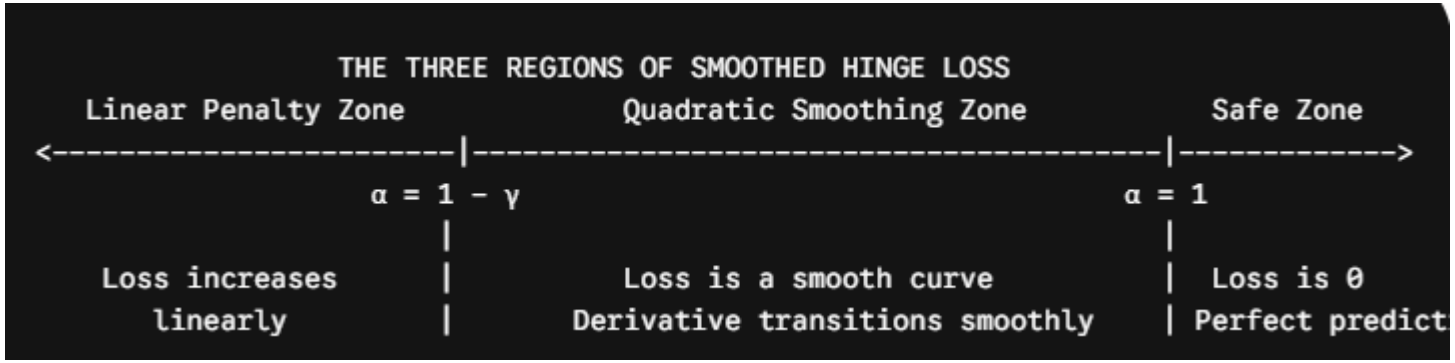
- Key difference with Hinge loss
  - Perceptron only penalizes wrong predictions.
  - Hinge also penalizes correct but "too close" predictions.
  - Example. Margin = 0.5 → Perceptron = 0, Hinge = 0.5
- Limitation of Hinge Loss
  - Large errors cause very large penalties, so extreme outliers can distort the model.
  - At margin = 1, hinge loss has a sharp corner (Not smoothly differentiable), which can make optimization harder. However, hinge loss is differentiable almost everywhere (consider 0.999 or 1.001) and subgradient method to randomly assign a slope between $[-1,0]$.

Loss function – Smoothed Hinge Loss

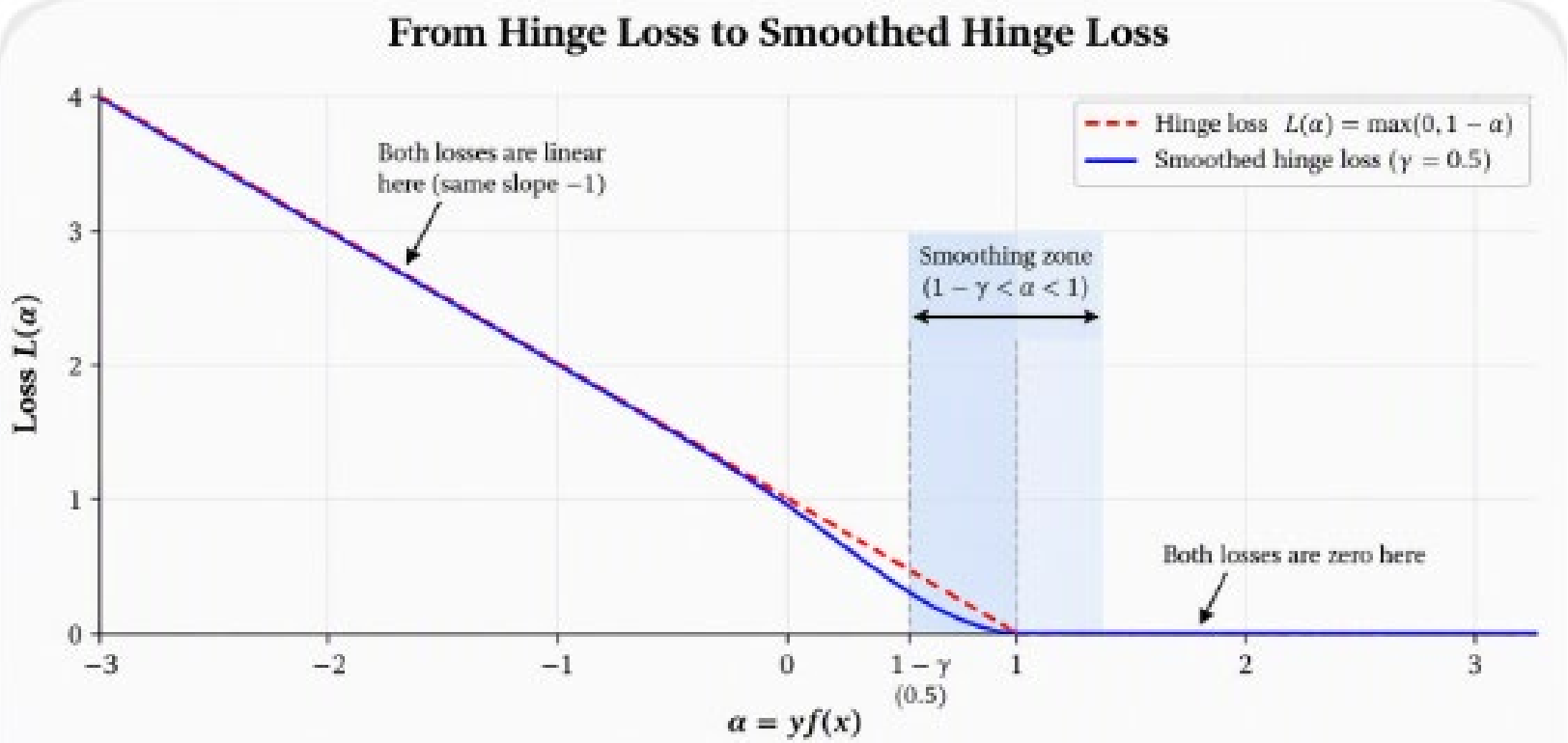
- When researcher requires strict, continuous differentiability, they swap out the standard Hinge loss for the Smoothed Hinge Loss.
- The Smoothed Hinge loss rounds off the sharp corner by inserting a quadratic piece-wise curve right at the transition boundary (parameterized by a small smoothing zone γ):

$$L(f(x), y) = \begin{cases} 0 & if\ \alpha \geq 1 \\ \dfrac{(1-\alpha)^2}{2\gamma} & if\ 1-\gamma < \alpha < 1 \\ 1 - \alpha - \dfrac{\gamma}{2} & if\ \alpha \leq 1-\gamma \end{cases}$$

where $\alpha = yf(x)$.



Loss function – Smoothed Hinge Loss



Loss function – Smoothed Hinge Loss

- The Safe Zone $(\alpha \geq 1)$
  - When the prediction is correct with $yf(x) \geq 1$, the loss is 0. Its derivative is also 0.
- The Linear Penalty Zone $(\alpha \leq 1 - \gamma)$
  - When the model is significantly wrong or too close to the boundary, the loss increases linearly: $1 - \alpha - \frac{\gamma}{2}$. If we take the derivative of this with respect to the margin α, the slope is -1.
- The Quadratic Smoothing Zone $(1 - \gamma < \alpha < 1)$
  - An interpolation zone where the slope $-\frac{1-\alpha}{\gamma}$ changes from -1 $(\alpha = 1 - \gamma)$ to 0 $(\alpha = 1)$.
  - Because the quadratic curve's slope matches the surrounding zones at the stitch points, the gradient changes smoothly from -1 to 0 as the margin increases from $1 - \gamma$ to 1.

Loss function – Smoothed Hinge Loss

- The parameter γ controls the smoothness and the optimization stability.
  - Large γ – when γ is increased, the smoothing window is widened. The gradient changes gradually across a large range of margin values. It prevents sudden, erratic weight updates when data points cross the margin, allowing the algorithm to train with higher learning rates without numerical instability or oscillations.
  - $\gamma \to 0$ (Convergence to Standard Hinge Loss) – As γ shrinks to zero, the quadratic window collapses. The smoothing region shrinks to a single point. The function converges to the standard hinge loss, and the gradient becomes sharp and abrupt (either 0 or −1), bringing back the non-differentiable corner.

Probabilistic Loss Function

- Probabilistic losses treat:
  - The **real data** as a true probability distribution
  - The **model** as an estimated distribution
  - Training means adjusting the model (parameters) so its predicted probabilities match the true ones. Unlike margin-based methods, this gives **interpretable outputs** (e.g., "80% confidence").
- For binary classification ($y \in \{0,1\}$):
  - The model predicts $f(x) = P(Y = 1 \mid x)$
  - The dataset likelihood is: $\prod_i f(x_i)^{y_i}\big(1 - f(x_i)\big)^{1-y_i}$
  - Taking logs turns this into a sum and avoids numerical issues:

$$\sum [\, y_i \log f\,(x_i) + (1 - y_i)\log\big(1 - f(x_i)\big)]$$

  - Negating it gives the **Negative Log-Likelihood (NLL)** loss.

Probabilistic Loss Function

- Cross-entropy measures how different two probability distributions are.
  For classification, it becomes:

$$-\sum[\,y_i \log f\,(x_i) + (1-y_i)\log\big(1-f(x_i)\big)]$$

  This is exactly the same as NLL. So, maximizing likelihood = minimizing cross-entropy.

Probabilistic Loss Function

- For multiple classes:
  - Use **Softmax** to turn raw scores into probabilities. If $f_k(x)$ is a score for class k, its softmax function output is defined as

$$f_S\left(f_k(x)\right) = \frac{e^{f_k(x)}}{\sum_{j=1}^{K} e^{f_j(x)}}, k \in \{1,2,\dots,K\}.$$

  - Apply **Categorical Cross-Entropy**:

$$L = -\sum_{j=1}^{K} y_j \log f_S\left(f_j(x)\right)$$

  - Here $y = (y_1, y_2, \dots, y_K)$ is the true label represented as a one-hot encoded vector.
  - In a 3-class problem (Cat, Dog, Bird), if the true animal is a Dog, the label vector is $y = (0,1,0)$, that is $y_1 = 0$ (Cat), $y_2 = 1$ (Dog) and $y_3 = 0$ (Bird).

Probabilistic Loss Function

- For multiple classes:
  - Let's expand the summation using this vector:

$$L = -\left[y_1 \log f_S\left(f_1(x)\right) + y_2 \log f_S\left(f_2(x)\right) + y_3 \log f_S\left(f_3(x)\right)\right]$$
$$= -[0 \cdot \log P(\text{Cat}) + 1 \cdot \log P(\text{Dog}) + 0 \cdot \log P(\text{Bird})] = -\log P(\text{Dog})$$

  - Because the true label vector contains zeros everywhere except at the correct index, every incorrect class gets multiplied by 0 and vanishes. The equation simplifies to just the true class index (let's call it c):

$$L = -\log f_S\left(f_c(x)\right) = -\log(\text{probability of correct class})$$

    - High probability → If the model assigns a 99% probability to the correct class, the loss is $-\log(0.99) \approx 0.01$. The loss (penalty) is small.
    - Low probability → If the model assigns a 1% probability to the correct class, the loss is $-\log(0.01) \approx 4.60$. The loss (penalty) is large.

Bias-Variance Trade-off

- In linear regression model, the least square estimator is

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}}(Y - X\beta)'(Y - X\beta) = (X'X)^{-1}X'Y$$

- Criteria to check $\hat{\beta}$ is a good estimator:
  - How close $\hat{\beta}$ to the true β?
  - Will $\hat{\beta}$ predict future observations well?
- Consider the MSE of $\hat{\beta}$:

$$MSE(\hat{\beta}) = E\left[(\hat{\beta} - \beta)'(\hat{\beta} - \beta)\right] = E\left[tr\left((\hat{\beta} - \beta)(\hat{\beta} - \beta)'\right)\right] = tr\left(Var(\hat{\beta})\right) = \sigma^2 tr\left((X'X)^{-1}\right)$$

- For a new data (x_0, y_0),

$$EPE(x_0, y_0) = E\left[\left(y_0 - x_0'\hat{\beta}\right)^2\right] = E\left[\left(y_0 - x_0'\beta + x_0'\beta - x_0'\hat{\beta}\right)^2\right]$$

$$= E\left[(y_0 - x_0'\beta)^2 + x_0'(\hat{\beta} - \beta)(\hat{\beta} - \beta)'x_0 + 2(y_0 - x_0'\beta)\left(x_0'\beta - x_0'\hat{\beta}\right)\right] = \sigma^2 + \sigma^2 x_0'(X'X)^{-1}x_0$$

Bias-Variance Trade-off

- When the columns of X are highly correlated (multicollinearity), the matrix $(X'X)^{-1}$ becomes unstable. This leads to very large standard errors, meaning small changes in the data can cause big swings in the estimated coefficients $\hat{\beta}$.
- Another issue arises when the number of features exceeds the number of observations ($p > n$), in which case $(X'X)^{-1}$ may not even exist.
- To address this, we impose constraints on $\hat{\beta}$ (regularization), which stabilizes the estimates. Although this introduces some bias, it significantly reduces variance, leading to a lower overall mean squared error (MSE).
- In machine learning, regularization helps prevent overfitting by discouraging overly complex models and ensuring that predictions remain stable and reliable.

L1 Regularization

- Lasso (L_1) adds a penalty proportional to the **absolute sum** of the weights $(\lambda > 0)$:

$$\hat{\beta}_{\lambda}^{Lasso} = \underset{\beta}{\operatorname{argmin}} \left[(Y - X\beta)'(Y - X\beta) + \lambda \sum_{j=1}^{p} |\beta_j| \right]$$

  - $RSS = (Y - X\beta)'(Y - X\beta)$ measures how well the model fits the training data.
  - The L_1 penalty $\sum_{j=1}^{p} |\beta_j|$ measures the magnitude of the weights.
  - The parameter λ dictates the trade-off. If $\lambda = 0$, it is OLS. As λ increases to infinity, the penalty dominates, forcing all coefficients $\hat{\beta}$ shrink to zero.
  - When $X'X = I_{p+1}$, $\hat{\beta}^{OLS} = (X'X)^{-1}X'Y = X'Y$.

$$\min_{\beta} \left\{ (Y - X\beta)'(Y - X\beta) + \lambda \sum_{j=1}^{p} |\beta_j| \right\} = \min_{\beta} \left\{ -\beta' X'Y + \frac{1}{2}\beta'\beta + \lambda \sum_{j=1}^{p} |\beta_j| \right\}$$

$$= \min_{\beta} \left\{ -\beta' \hat{\beta}^{OLS} + \frac{1}{2}\beta'\beta + \lambda \sum_{j=1}^{p} |\beta_j| \right\} = \min_{\beta} \sum_{j=1}^{p} \left\{ -\beta_j \hat{\beta}_j^{OLS} + \frac{1}{2}\beta_j^2 + \lambda |\beta_j| \right\}$$

L1 Regularization

- Lasso (L_1) adds a penalty proportional to the **absolute sum** of the weights $(\lambda > 0)$:

$$\hat{\beta}_{\lambda}^{Lasso} = \underset{\beta}{\operatorname{argmin}} \left[(Y - X\beta)'(Y - X\beta) + \lambda \sum_{j=1}^{p} |\beta_j| \right]$$

  - Fix a j. We want to minimize $\mathcal{L}_j = -\beta_j \hat{\beta}_j^{OLS} + \frac{1}{2}\beta_j^2 + \lambda|\beta_j|$

    Case 1: $\hat{\beta}_j^{OLS} > 0$. We claim that $\beta_j > 0$ otherwise we can set $\beta_j = 0$ to obtain a lower $\mathcal{L}_j$ value.

    Set $\frac{\partial \mathcal{L}_j}{\partial \beta_j} = -\hat{\beta}_j^{OLS} + \beta_j + \lambda = 0$. We obtain $\hat{\beta}_j = \hat{\beta}_j^{OLS} - \lambda \geq 0$.

    Case 2: $\hat{\beta}_j^{OLS} \leq 0$. We claim that $\beta_j < 0$ otherwise we can set $\beta_j = 0$ to obtain a lower $\mathcal{L}_j$ value.

    Set $\frac{\partial \mathcal{L}_j}{\partial \beta_j} = -\hat{\beta}_j^{OLS} + \beta_j - \lambda = 0$. We obtain $\hat{\beta}_j = \hat{\beta}_j^{OLS} + \lambda \leq 0$.

  - When $X'X = I_{p+1}$, $\hat{\beta}_{\lambda,j}^{Lasso} = sign\left(\hat{\beta}_j^{OLS}\right)\left(\left|\hat{\beta}_j^{OLS}\right| - \lambda\right)_+$ where $(a)_+ = \max(0, a)$.

  - In general, there is no closed-form solution for $\hat{\beta}_{\lambda}^{Lasso}$.

L1 Regularization

- Lasso (L_1) constraint region is a polytope (a diamond with sharp corners) forces some weights to become exactly zero. Automatically performs feature selection which produces a simpler, more interpretable models.
- To find optimal λ, we use k-fold cross validation.
  - The training dataset is split into k equal segments (folds).
  - For a specific choice of λ, the model is trained on $k-1$ folds and evaluated on the remaining "validation" fold, calculating the Mean Squared Error (MSE).
  - This is repeated k times so every fold acts as the validation set once.
  - The k errors are averaged to give a single cross-validation error for that specific λ.
  - This process is run across a grid of potential λ values. We pick the λ that minimizes the average validation error.
- Limitation: Not smooth at zero – harder to optimize. To solve this in practice, machine learning libraries use alternative optimization strategies like Coordinate Descent or Proximal Gradient Methods to handle the sharp corners cleanly.

L1 Regularization (Bayesian Consideration)

- Assume prior $\beta_j \sim P(\beta_j) \propto \exp\left(-\frac{|\beta_j|}{b}\right), b > 0$ (This is called a Laplace prior)
- Taking log, $-\ln P(\beta) \propto \sum_{j=1}^{p} |\beta_j|$
- $P(Data|\beta) \propto \exp\left(-\frac{1}{2\sigma^2}(Y - X\beta)'(Y - X\beta)\right)$
- Taking log, $-\ln P(Data|\beta) \propto (Y - X\beta)'(Y - X\beta)$
- Since $P(\beta|Data) \propto P(Data|\beta)P(\beta)$,

$$-\ln P(\beta|Data) \propto -\ln P(Data|\beta) - \ln P(\beta) = (Y - X\beta)'(Y - X\beta) + \lambda \sum_{j=1}^{p} |\beta_j|$$

L2 Regularization

- Ridge (L_2) adds a penalty proportional to the **squared sum** of the weights $(\lambda > 0)$:

$$\hat{\beta}_{\lambda}^{Ridge} = \underset{\beta}{\operatorname{argmin}}[(Y - X\beta)'(Y - X\beta) + \lambda\beta'\beta] = \underset{\beta}{\operatorname{argmin}}[-2\beta'X'Y + \beta'X'X\beta + \lambda\beta'\beta]$$

$$= \underset{\beta}{\operatorname{argmin}}\left[-2\beta'X'Y + \beta'\left(X'X + \lambda I_{p+1}\right)\beta\right] = \left(X'X + \lambda I_{p+1}\right)^{-1}X'Y$$

- Note that $X'X + \lambda I_{p+1}$ is always invertible.
- The constraint region for Ridge is a smooth hypersphere $\left(\sum_{j=1}^{p} \beta_j^2 \leq t\right)$. Thus, all features retain a small, non-zero weight.
- Since $\frac{\partial\left(\lambda\beta_j^2\right)}{\partial\beta_j} = 2\lambda\beta_j$, the size of the regularization update is proportional to β_j during gradient descent $\left(\hat{\beta}_j \leftarrow \hat{\beta}_j - \rho\frac{\partial\mathcal{L}_j}{\partial\beta_j}\right)$.
  - If β_j is large, the penalty pulls it down aggressively.
  - As β_j gets close to zero, the derivative becomes negligibly small.

L2 Regularization

- The penalty forces decay to nothing as the weight approaches zero, meaning it will get infinitely close to zero, but will never be zero. Shrinks weights toward zero but rarely makes them exactly zero.
- The parameter λ controls the level of overfitting.
  - $\lambda = 0$ no regularization
  - $\hat{\beta} \to 0$ as $\lambda \to \infty$
- When we have two or more correlated features,
  - Lasso will arbitrarily pick one of the correlated features, give it the full coefficient weight, and set the other to zero.
  - Ridge hates large weights because squaring them heavily amplifies the penalty. To minimize the overall loss, Ridge shares the burden by distributing the coefficient weight evenly across all correlated features, shrinking their individual magnitudes together.

L2 Regularization

- λ is a hyperparameter selected via k-fold cross-validation. We sweep across a range of log-spaced λ values, evaluate the model on validation folds, and choose the λ that minimizes the average validation Mean Squared Error (MSE).
- Because the squared penalty is a quadratic function, the entire objective function is strictly convex, smooth, and continuously differentiable everywhere. This makes Ridge exceptionally easy and fast to optimize using standard numerical linear algebra solvers or basic Gradient Descent, as the gradient is clean and well-defined at every single point in the parameter space.

L2 Regularization (Bayesian Consideration)

- Assume prior $\beta_j \sim n(0, \tau^2)$ (This is called a Gaussian prior)
- Taking log, $-\ln P(\beta) \propto \sum_{j=1}^{p} \beta_j^2$
- $P(Data|\beta) \propto \exp\left(-\frac{1}{2\sigma^2}(Y - X\beta)'(Y - X\beta)\right)$
- Taking log, $-\ln P(Data|\beta) \propto (Y - X\beta)'(Y - X\beta)$
- Since $P(\beta|Data) \propto P(Data|\beta)P(\beta)$,

$$-\ln P(\beta|Data) \propto -\ln P(Data|\beta) - \ln P(\beta) = (Y - X\beta)'(Y - X\beta) + \lambda \sum_{j=1}^{p} \beta_j^2$$

Regularization on smooth function

- Consider estimation of an arbitrary function f. The MSE is given by

$$MSE(f) = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - f(x_i)\big)^2$$

- Minimizing the MSE leads to infinitely many solutions since any function g that passes through the data points $\{(x_i, y_i), i = 1,2, \dots, n\}$ is a solution.
- To reduce overfitting, we add a Spline penalty for non-smoothness.

$$PRSS(f;\lambda) = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - f(x_i)\big)^2 + \lambda\int[f''(x)]^2 dx$$

- $\lambda = 0$ no regularization
- $\lambda = \infty$ only linear function in x is permitted $(f''(x) = 0)$.

Regularization on smooth function

- The Spline penalty term penalizes large values of the second derivative much more harshly than small one.
- It smoothly dampens the variation of the function everywhere, thus controlling curvature of the function.
- $f'(x)$ represents slope, $f''(x)$ curvature (how fast the slope changes)
- If the function $f(x)$ oscillates wildly over a small region to hit noisy data points, its second derivative $[f''(x)]^2$ will explode, making the integral massive. The penalty forces the function to stay smooth.
- Instead of searching over infinite possible functions, we represent $f(x) = \sum_j \alpha_j h_j(x)$ where $h_j(x)$s are known basis functions.

Regularization on smooth function

- When we substitute this back into the penalty integral, we have

$$\int [f''(x)]^2 dx = \int \left(\sum_{j=1}^{m} \alpha_j h_j''(x) \right) \left(\sum_{k=1}^{m} \alpha_k h_k''(x) \right) dx = \sum_{j=1}^{m} \sum_{k=1}^{m} \alpha_j \alpha_k \underbrace{\int h_j''(x) h_k''(x) dx}_{\Omega_{j,k}} = \alpha' \Omega \alpha$$

  where $\Omega = [\Omega_{j,k}]$ is an $m \times m$ matrix of inner product

$$\Omega_{j,k} = \int h_j''(x) h_k''(x) dx$$

  and $\alpha = (\alpha_1, \alpha_2, \ldots, \alpha_m)'$

- The optimizing function becomes $PRSS(\alpha) = (Y - H\alpha)'(Y - H\alpha) + \lambda \alpha' \Omega \alpha$. Hence,

$$\hat{\alpha} = (H'H + \lambda\Omega)^{-1} H'Y$$

Regularization on smooth function

Example

Suppose we observe

x_i	0	1	2	3	4
y_i	1	2	4	3	5

and approximate the unknown function using the quadratic basis $h_1(x) = 1, h_2(x) = x, h_3(x) = x^2$.

Thus, $f(x) = \alpha_1 + \alpha_2 x + \alpha_3 x^2$. The first and second derivatives are $f'(x) = \alpha_2 + 2\alpha_3 x$ and $f''(x) = 2\alpha_3$.

Suppose we measure roughness over $0 \leq x \leq 4$. The spline penalty is

$$\int_0^4 [f''(x)]^2 \, dx.$$

Since $f''(x) = 2\alpha_3$,

$$\begin{aligned} \int_0^4 [f''(x)]^2 dx &= \int_0^4 (2\alpha_3)^2 dx \\ &= \int_0^4 4\,\alpha_3^2 dx \\ &= 16\alpha_3^2. \end{aligned}$$

Regularization on smooth function

Example

x_i	0	1	2	3	4
y_i	1	2	4	3	5

Therefore, $\boldsymbol{\alpha}'\Omega\boldsymbol{\alpha} = 16\alpha_3^2$, where

$$\Omega = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 16 \end{pmatrix}.$$

The design matrix is

$$H = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 4 & 16 \end{pmatrix},$$

and

$$Y = \begin{pmatrix} 1 \\ 2 \\ 4 \\ 3 \\ 5 \end{pmatrix}.$$

By choosing an appropriate λ, we have
$\hat{\alpha} = (H'H + \lambda\Omega)^{-1}H'Y$